# Centralized Database Systems

## 1 Introduction

A **database system architecture** defines how a database system is structured and how its components interact with each other. One of the earliest and most widely used architectures is the **Centralized Database System**.

In this model, all data, database software, and control mechanisms are located on a **single computer system**. Despite the rise of distributed and cloud databases, centralized systems are still widely used due to their simplicity, consistency, and ease of management.

## 2 Core Concepts

**Centralized Database System**

A system in which the entire database and the **DBMS** reside on a single computer system, and all users access the data through that system.

This single system may range from:

- A small personal computer
- A powerful enterprise server
- A high-performance multi-core machine

### 2.1 Key Characteristics

- **Single Location:** All data is stored in one physical location.
- **Central Control:** One DBMS controls concurrency, recovery, and security.
- **Multiple Users:** Can support tens to thousands of users simultaneously.
- **Shared Resources:** CPU, memory, and disks are shared by all users.

## 3 Types of Centralized Systems

### 3.1 1. Single-User Centralized Systems

- Designed for one user at a time.
- Minimal concurrency control.
- Often accessed using APIs instead of SQL.

**Embedded Use Case**

**SQLite** used inside mobile apps or desktop software is a classic example of a single-user centralized database.

### 3.2 2. Multi-User Centralized Systems (Server Systems)

- Support multiple concurrent users via a **client–server model**.
- Strong concurrency control and recovery mechanisms.

## 4 Server System Architecture

Centralized database servers typically follow a **transaction server architecture** (also known as a query server).

**Transaction Server Mechanics**

- Clients send SQL requests to the server.
- Transactions execute entirely on the server.
- Only results are sent back to clients.
- Standard APIs like **ODBC** or **JDBC** are used for communication.

## 5 Internal DBMS Components

### 5.1 Shared Memory Area

All server processes access a common shared memory containing:

- **Buffer Pool:** Cached disk pages for fast access.
- **Lock Table:** Tracking active locks for concurrency.
- **Log Buffer:** Staging log records before disk writes.

### 5.2 Process Management

- **Server Processes:** Handle SQL execution and client communication.
- **Database Writer:** Writes modified buffers to disk.
- **Log Writer:** Writes log records to stable storage.
- **Checkpoint Process:** Synchronizes memory and disk states for recovery.

## 6 Concurrency and Control

To ensure consistency, centralized systems use **atomic instructions** to manage a shared lock table in memory.

- **Test-and-Set:** Used to acquire a mutex (mutual exclusion).
- **Compare-and-Swap (CAS):** Used for conditional atomic updates.

## 7 Advantages and Limitations

**Why use Centralized Systems?**

- **Strong Consistency:** Single source of truth.
- **Simplicity:** Easier backup, recovery, and administration.
- **No Network Latency:** Internal data access avoids network bottlenecks.

**The Scale Wall**

- **Single Point of Failure:** System crash brings everything down.
- **Scalability:** Limited by the hardware of the single machine.
- **Response Time:** Can degrade under extreme concurrent load.

*Centralized systems remain a foundation for many applications, providing a balance of simplicity and reliability for non-distributed needs.*